1 Material and Methods

This section describes the materials, software environment, data preparation procedure, retrieval modules, prediction branches, and evaluation protocol used in the proposed dermoscopic image classification system. The system was implemented as a Python-based pipeline that combines image retrieval, optional text retrieval, local large language model inference, local vision-language model inference, and automatic metric generation. The pipeline performs metadata ingestion, image matching, train-test splitting, train-only augmentation, embedding-index construction, retrieval-based prediction, generative prediction, and branch-wise evaluation.

1.1 Materials

1.1.1 Dataset and Input Structure

write later

1.1.2 Software Environment

The system was implemented in Python. Data handling was performed using `pandas` and `NumPy`. Image loading, RGB conversion, and augmentation were performed using `Pillow`. Train-test splitting, classification reports, confusion matrices, accuracy, balanced accuracy, and F1-score computation were implemented using `scikit-learn`. Plot generation was performed using `matplotlib`. The default image encoder is CLIP, loaded through the Hugging Face `transformers` library. The default text encoder is `sentence-transformers/all-MiniLM-L6-v2`. Optional fast similarity search is supported through FAISS. Local LLM and VLM inference is performed through Ollama.

The default local text model is `llama3.1:8b`, and the default local vision-language model is `llama3.2-vision:11b`. The system also supports lightweight fallback modes for offline testing. In fallback mode, image embeddings can be replaced by simple color-histogram features, and text embeddings can be replaced by TF-IDF features.

Table 1: Main software components used in the implementation.

Software Component	**Purpose**
Python	Main implementation language.
`pandas` and `NumPy`	Metadata processing, numerical operations, and table manipulation.
`Pillow`	Image loading, RGB conversion, and augmentation.
`scikit-learn`	Dataset splitting and classification metric computation.
`matplotlib`	Confusion-matrix and plot generation.
CLIP	Default visual embedding model for dermoscopic image retrieval.
Sentence Transformers	Default text embedding model for local text retrieval.
FAISS	Optional fast nearest-neighbor similarity search.
Ollama	Local LLM and VLM inference backend.

1.1.3 Runtime and Hardware Support

The pipeline automatically detects whether CUDA is available. If a GPU is detected, the system enables GPU execution for supported embedding models and uses automatic mixed precision where

applicable. Separate batch-size settings are defined for CPU, T4, A100, and H100 devices. For large GPUs, the image and text embedding batch sizes are increased to improve throughput. The implementation also contains automatic batch-size adjustment and memory-cleaning routines to reduce out-of-memory failures during embedding generation.

1.2 Methods

1.2.1 Overall Pipeline Workflow

The proposed method follows a retrieval-centered diagnostic workflow. First, the metadata table is loaded and normalized. Column names are stripped of extra whitespace, the `isic_id` field is validated, missing values are standardized, and boolean-like values in the `melanocytic` column are converted into consistent textual labels. Second, image files are recursively collected from the image directory and matched with metadata records. Third, unmatched metadata rows are removed from the evaluation dataset.

The matched dataset is divided into training and testing partitions using a fixed random seed of 42. The configured training ratio is 0.80, producing an 80:20 train-test split. Stratification is attempted using `diagnosis_1` when sufficient class counts are available. If stratification is not possible, the system falls back to a standard shuffled random split.

After splitting, augmentation is applied only to the training set. The test set is never augmented. The augmented training set is then embedded and indexed. Test images are embedded using the same image encoder, and nearest training examples are retrieved for each test image. These retrieved cases are used as evidence by the weighted kNN, VLM-RAG, and LLM-RAG branches.

The complete execution flow is as follows:

1. Load metadata, dermoscopic images, and optional local text corpus.
2. Match image files to metadata records using the `isic_id` field.
3. Remove unmatched metadata rows.
4. Split matched data into training and testing partitions.
5. Apply train-only image augmentation.
6. Build image embeddings for the training set.
7. Construct a nearest-neighbor retrieval index over training embeddings.
8. Embed test images and retrieve top visual neighbors.
9. Run selected prediction branches: weighted kNN, VLM-RAG, and LLM-RAG.
10. Save predictions, errors, evaluation metrics, plots, and run artifacts.

1.2.2 Manifest-Based Reproducibility

To prevent stale cached outputs from being reused after changes in the input data or configuration, the system creates a run manifest. The manifest records the selected branches, image backend, text backend, local model names, metadata hash, text-corpus hash, image count, image-listing hash, and configuration values. At the beginning of each run, the new manifest is compared with the previous manifest. If any tracked input or configuration has changed, cached outputs, prediction files, metric files, plots, and intermediate artifacts are cleared before execution. This mechanism ensures that the generated results correspond to the current data and configuration.

1.2.3 Train-Only Image Augmentation

The training set is expanded using deterministic image augmentation. For each original training image, one augmented copy is generated by default. The augmentation operations include horizontal flipping, occasional vertical flipping, random rotation, small random cropping, brightness jitter, contrast jitter, color jitter, sharpness jitter, and JPEG re-encoding. The maximum rotation angle is 15 degrees, and augmented images are saved with JPEG quality 95.

Each augmented sample receives a new identifier while preserving its source image identifier. The random seed for augmentation is computed from the global seed, image identifier, augmentation index, and original filename. This makes the augmentation process reproducible across runs. Since augmentation is applied only after the train-test split and only to training images, the procedure avoids test-set leakage.

1.2.4 Image Embedding and Visual Retrieval

The default image representation is obtained using the CLIP image encoder `openai/clip-vit-base-patch32`. Each image is converted to RGB, processed by the CLIP processor, passed through the image encoder, and normalized to unit length. Let x_q denote a query image and $f_I(\cdot)$ denote the image encoder. The normalized query embedding is computed as

$$z_q = \frac{f_I(x_q)}{\|f_I(x_q)\|_2}. \tag{1}$$

For every indexed training image x_i, the stored embedding is

$$z_i = \frac{f_I(x_i)}{\|f_I(x_i)\|_2}. \tag{2}$$

The similarity between the query image and a training image is computed using inner product:

$$s(q, i) = z_q^T z_i. \tag{3}$$

Because all vectors are normalized, this inner product is equivalent to cosine similarity. The top-k visually similar training images are retrieved for each query image, with $k = 5$ by default. FAISS is used for efficient similarity search when available. If FAISS is unavailable, the system performs direct NumPy-based similarity computation.

When the CLIP backend is unavailable, the system falls back to a simple image encoder based on RGB color histograms. This fallback is intended for smoke testing and offline execution rather than final experimental reporting.

1.2.5 Text Chunking and Text Retrieval

The optional local text corpus is used only in the LLM-RAG branch. Text is loaded from plain-text, markdown, CSV, or spreadsheet files. After loading, whitespace is normalized and the text is divided into overlapping word chunks. The default chunk length is 180 words, with an overlap of 40 words between consecutive chunks.

Each chunk is embedded using the default sentence-transformer model. Let t_j denote a text chunk and $f_T(\cdot)$ denote the text encoder. The normalized chunk embedding is given by

$$u_j = \frac{f_T(t_j)}{\|f_T(t_j)\|_2}. \tag{4}$$

During inference, a text query is constructed from patient metadata and the labels of retrieved visual neighbors. The query is embedded using the same text encoder and compared against the stored chunk embeddings. The top retrieved text chunks are inserted into the LLM prompt as external context. The default number of retrieved text chunks is four.

If the sentence-transformer backend is unavailable, the text branch falls back to a TF-IDF representation. The TF-IDF vectors are also normalized before similarity search.

1.2.6 Weighted kNN Prediction Branch

The weighted kNN branch provides a transparent non-generative baseline. For each test image, the system retrieves its top visual neighbors from the training index. For each target variable, every neighbor contributes a vote weighted by its visual similarity score. For a target label y, the prediction is computed as

$$\hat{y} = \arg\max_{c} \sum_{i \in \mathcal{N}_k(q)} s(q,i)\, \mathbf{1}(y_i = c), \tag{5}$$

where $\mathcal{N}_k(q)$ is the set of top-k nearest training images for query image q, $s(q,i)$ is the similarity between the query and neighbor i, and $\mathbf{1}(y_i = c)$ is an indicator function. If no valid label is available for a target, the prediction is recorded as null.

1.2.7 Vision-Language Model Retrieval-Augmented Branch

The VLM-RAG branch uses a local vision-language model through Ollama. The default model is `llama3.2-vision:11b`. For each test sample, the query image is converted to base64 format and submitted to the Ollama chat API. The VLM prompt contains the active target names, the allowed label vocabulary, patient metadata when available, and retrieved visually similar training cases.

This branch deliberately excludes external medical text. Therefore, the VLM-RAG branch uses only the image and visual-neighbor evidence. The model is instructed to return a compact JSON object containing only the requested target fields. The generated output is then sanitized by checking whether each predicted value belongs to the training-set label vocabulary. Invalid labels are replaced with null.

1.2.8 Language Model Retrieval-Augmented Branch

The LLM-RAG branch uses a local text-only LLM through Ollama. The default model is `llama3.1:8b`. This branch combines patient metadata, retrieved visual-neighbor cases, retrieved medical text chunks, and optionally the previous VLM prediction. The VLM prediction is treated only as auxiliary evidence and not as ground truth.

The LLM prompt is constructed using the following components:

1. active prediction targets;
2. allowed label vocabulary derived from the training set;
3. available patient or metadata context;
4. top visually similar training cases;
5. retrieved text chunks from the optional corpus;
6. optional VLM prediction for the same sample.

The model is instructed to return JSON matching a fixed schema. The system parses the returned JSON and performs post-processing to ensure that all predicted labels belong to the allowed training vocabulary. Labels outside the vocabulary are replaced with null. This strict post-processing step prevents free-form model outputs from entering the evaluation.

Table 2: Prediction branches implemented in the proposed system.

Branch	**Evidence Used**	**Prediction Mechanism**
kNN	Query image and retrieved visual neighbors	Similarity-weighted voting over training labels.
VLM-RAG	Query image, metadata context, and retrieved visual-neighbor evidence	Local vision-language model predicts structured target labels.
LLM-RAG	Metadata context, retrieved visual neighbors, retrieved text chunks, and optional VLM output	Local text LLM predicts structured labels using retrieval-augmented context.

1.2.9 Local Model Validation and Error Handling

Before inference, the system validates the requested execution branches. If local LLM or VLM branches are requested, the system checks whether the Ollama server is reachable. If the server is unavailable, the system can attempt to install Ollama locally, start the Ollama server, and pull missing models. Requested local models are warmed up before prediction begins. If a requested local branch cannot be activated, the system raises an error by default instead of silently removing the branch. This behavior prevents accidental reporting of incomplete branch comparisons.

Generative predictions are cached using a hash of the branch name, image identifier, prompt, model name, and image hash. This avoids repeated model calls for unchanged samples. If an Ollama call fails, the request is retried according to the configured retry policy. If all retries fail, the corresponding prediction is recorded with null labels, and the error message is written to a branch-specific error file. This allows long-running evaluations to continue while preserving failure information for later analysis.

1.2.10 Evaluation Metrics

Each prediction branch is evaluated separately. For every active target, rows with non-null ground-truth labels and non-null predictions are scored. The reported metrics include accuracy, macro-F1, weighted-F1, and balanced accuracy. Confusion matrices are generated for each branch-target pair and saved as image files.

Accuracy is defined as

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{y}_i = y_i), \tag{6}$$

where N is the number of evaluated samples, y_i is the ground-truth label, and $\hat{y}_i$ is the predicted label.

For class c, precision and recall are computed as

$$P_c = \frac{TP_c}{TP_c + FP_c}, \tag{7}$$

$$R_c = \frac{TP_c}{TP_c + FN_c}, \tag{8}$$

where TP_c, FP_c, and FN_c denote true positives, false positives, and false negatives for class c, respectively. The class-wise F1-score is

$$F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{9}$$

Macro-F1 is computed by averaging $F1_c$ equally across classes:

$$\text{MacroF1} = \frac{1}{C}\sum_{c=1}^{C} F1_c, \tag{10}$$

where C is the number of classes. Weighted-F1 computes the same average while weighting each class by its support. Balanced accuracy is computed as the average recall across classes:

$$\text{BalancedAccuracy} = \frac{1}{C}\sum_{c=1}^{C} R_c. \tag{11}$$

These metrics are used because dermoscopic datasets may contain class imbalance. Macro-F1 and balanced accuracy provide a more informative estimate of minority-class performance than overall accuracy alone.

1.2.11 Output Artifacts

After final results